# FEAT-ZD-48600 · DE-07 Identify the sources/pipelines to be changed (Transformation) ■ · Components, layers, sources or pipelines to change

Data Engineering agent · item task.components · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.components.md · approved by Data engineer sim.data_engineer

## task.components: FEAT-ZD-48600

Sources and pipelines to create or update in the **transformation pipeline** for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DE-07, Identify the sources/pipelines to be changed (Transformation), attempt 3. It builds on the DE-04 ingestion list (task.components, a126bc4) and the DE-06 mapping (feat.mappings, 2e9108d), and replaces the DE-04 version of this file. There is no gate: automated checks, the independent judge and sampled human evaluation review this output (record:brief).

| Field | Value | Source |
|---|---|---|
| Trace ID | FEAT-ZD-48600 | record:brief |
| Step | DE-07 Identify the sources/pipelines to be changed (Transformation), attempt 3 | record:brief |
| Inputs | feat.mappings (2e9108d, from DE-06), task.components (a126bc4, from DE-04) | record:brief |
| Code read | ap_pipeline `__init__.py`, `ingest.py`, `transform.py`, `dq.py`, `pipeline.py`, `schedule.json`, `tests/test_pipeline.py`, as given in the step's tool view | tool:repo |
| Repository / branch | ap_pipeline, branch `feature/story`; CI `/usr/bin/python3 ci.py` | task.components (header, section 4) |
| **Result** | **Two transformation components change:** `transform.to_model` **is updated additively (new** `paid_date` **mapping, TR-1) and one new pure module** `delays.py` **is created (**`delays_by_vendor`**, TR-2).** Ingestion, data quality, orchestration (`pipeline.run`) and the schedule need no code change. No new source, no new pipeline, no storage and no secret | feat.mappings (sections 2, 3, 4, 5, 7); task.components (sections 2, 3, 4); tool:repo |

### 1. Transformation pipeline today (read from the repository)

| Element | Stage | What it does today | Relevant to this feature | Source |
|---|---|---|---|---|
| `ingest.parse_feed(csv_text)` | Ingest | `csv.DictReader` rows as `dict`, keyed by the header text | Already carries a v3 `paid_date` column as a key; a v2 feed has no such key | tool:repo (`ingest.py`); task.components (IN-1, IN-2) |

| Element | Stage | What it does today | Relevant to this feature | Source |
|---|---|---|---|---|
| `transform.to_model(rows)` | Transform | One `try` block per row builds and appends `{"id", "vendor", "due_date", "amount", "paid"}`; on `KeyError`, `ValueError`, `InvalidOperation` or `AttributeError` it appends `{"row": r, "reason": type(e).__name__}` to `rejected`. Returns `(records, rejected)` | **Changes** (TR-1): no `paid_date` today | tool:repo (`transform.py`); feat.mappings (section 7) |
| `dq.quality_report(records, rejected)` | Data quality | `rows`, `loaded`, `rejected`, sorted set of `reject_reasons` | The new reason `paid_date` appears without a code change | tool:repo (`dq.py`); feat.mappings (section 3, step 7) |
| `pipeline.run(csv_text)` | Orchestration | `to_model(parse_feed(csv_text))`, returns `(records, quality_report(...))` | No change; `delays_by_vendor` is called by the caller, not by `pipeline.run` | tool:repo (`pipeline.py`); task.components (IN-3) |
| Delay history | Derived data | Does not exist; no `delays.py` module | **Created** (TR-2) | tool:repo; feat.mappings (sections 4, 7) |
| `schedule.json` | Schedule | `daily`, `02:00`, `Asia/Kolkata` | No change; matches the customer contract | tool:repo (`schedule.json`); feat.mappings (section 5) |
| `__version__` in `src/ap_pipeline/__init__.py` | Metadata | `0.4.0` | Version step only, in the TSK-01 budget | tool:repo (`__init__.py`); task.components (TC-1) |

## 2. Transformation change list (the outcome of this step)

| # | Source / pipeline | Action | Minimum change | Why this is the minimum | Source |
|---|---|---|---|---|---|
| TR-1 | `transform.to_model` (TC-1, TSK-01) | **Update, additive** | (a) Add constant `PAID_DATE_REASON = "paid_date"` and private helper `_parse_paid_date(row)`: `row.get("paid_date")`; `None` or blank after `strip()` → `None`; otherwise `date.fromisoformat` of the stripped value. (b) In the loop, build the existing five-field record in the existing `try` exactly as today, but keep it in a local instead of appending it; then, only when the `try` succeeded (for example in its `else:` branch), call `_parse_paid_date`; on `ValueError` append `{"row": r, "reason": PAID_DATE_REASON}` and skip the record; otherwise set `paid_date` and append the record | The five existing field rules and the `except` tuple stay as they are, so today's reject reasons win over `paid_date` (bad `due_date` + bad `paid_date` → `ValueError`). `paid_date` must be parsed **outside** the existing `try`: inside it, an invalid value would be rejected as `ValueError`, not `paid_date`. The `to_model` signature and `(records, rejected)` return shape stay unchanged | feat.mappings (sections 2, 3 steps 2–5, 6); task.components (TC-1, section 3 rules); tool:repo (`transform.py`) |
| TR-2 | `delays.delays_by_vendor(records)` (TC-2, TSK-02) | **Create** new module `src/ap_pipeline/delays.py` | One pure function, no I/O: for records where `paid` is truthy and `paid_date` is not `None`, append `(paid_date - due_date).days` (int, may be negative) under the record's `vendor`; vendors in first-seen order; returns `Dict[str, List[int]]`; does not modify the records | New behaviour goes into a new function and module; no change to `pipeline.run`, so its contract stays as is | feat.mappings (section 4, section 3 step 8); task.components (TC-2, IN-3); record:memory (D-2026-031) |
| TR-3 | `dq.quality_report` | **No change** | None | `reject_reasons` is the sorted set of each rejected row's `reason`, so `paid_date` is listed on its own | feat.mappings (section 3 step 7); task.components (IN-4); tool:repo (`dq.py`) |

| # | Source / pipeline | Action | Minimum change | Why this is the minimum | Source |
|---|---|---|---|---|---|
| TR-4 | `pipeline.run` | **No change** | None | Records gain `paid_date` inside `to_model`; the new reason reaches the report through TR-3; `delays_by_vendor` is called by the caller (end-to-end harness) | task.components (IN-3); tool:repo (`pipeline.py`) |
| TR-5 | `ingest.parse_feed` / vendor feed source (v2 and v3) | **No code change** | None | Header-driven; v2 rows lack the key and short v3 rows give `None`, both handled by `row.get("paid_date")` in TR-1 | task.components (IN-1, IN-2); feat.mappings (section 3 step 1, MQ-C) |
| TR-6 | `schedule.json` | **No change** | None | Customer contract daily 02:00 Asia/Kolkata equals the config | feat.mappings (section 5); task.components (IN-5) |
| TR-7 | New source, pipeline, storage or schema | **None created** | None | The mapping adds no new source, no new target entity and no storage change | feat.mappings (Result); task.components (IN-6) |

Rules carried for coding (DE-08): keep the `to_model` signature and return shape; do not add `strip()` to `due_date` parsing; `amount` stays `Decimal`, never float; every accepted record has exactly the keys `id`, `vendor`, `due_date`, `amount`, `paid`, `paid_date`; record-level rules are unchanged (a paid record with `paid_date = None` is kept, an unpaid record with a valid `paid_date` is kept, a `paid_date` before `due_date` is kept); no `__all__` in `ap_pipeline/__init__.py`, import as `from ap_pipeline.delays import delays_by_vendor` (feat.mappings sections 2, 3 step 6, 6; task.components section 3; record:memory D-2026-044, L-011).

## 3. Files in the change budget

| File | Change | Component | Source |
|---|---|---|---|
| `src/ap_pipeline/transform.py` | Update (additive) | TR-1 | task.components (TC-1) |
| `src/ap_pipeline/delays.py` | New | TR-2 | task.components (TC-2) |
| `src/ap_pipeline/__init__.py` | Version step only (from `0.4.0`) | TR-1 | task.components (TC-1, section 1); tool:repo (`__init__.py`) |
| `tests/test_pipeline.py` | Add the mapping validation cases of feat.mappings section 6 (v2 header, v3 blank, v3 valid, v3 invalid, bad `due_date` + bad `paid_date`, exact key set) on synthetic rows; existing tests stay | TR-1 | feat.mappings (section 6); task.components (TC-1); tool:repo (`tests/test_pipeline.py`) |
| `tests/test_delays.py` | New: filter on `paid` and `paid_date`, negative delays, first-seen vendor order, records not modified | TR-2 | feat.mappings (section 4); task.components (TC-2) |

Files that must not change: `ingest.py`, `dq.py`, `pipeline.py`, `schedule.json` (sections 1 and 2; task.components IN-2 to IN-5).

## 4. Secrets access in all environments

| Environment | Secrets required for the transformation pipeline | Granted | Gap | Source |
|---|---|---|---|---|
| Local development and CI (`/usr/bin/python3 ci.py` on `feature/story`) | None | None | None | task.components (section 4) |
| Integration (`dev`) and released (`main`) | None | None | None | task.components (section 4) |

TR-1 and TR-2 are pure functions over in-memory rows and records; the repository holds no feed connection, credential or workspace, and coding uses synthetic v2 and v3 rows only (tool:repo; task.components section 4; feat.mappings section 6). If releasing feed v3 later needs a credential, DE-02 runs again on that re-entry trigger (task.components section 4).

## 5. Dependencies and risks

| ID | Item | Handling | Source |
|---|---|---|---|
| R-T1 | Parsing `paid_date` inside the existing `try` would report an invalid value as `ValueError`, not `paid_date`, and break the reason precedence | TR-1 parses it after the `try` succeeds; the "bad `due_date` + bad `paid_date` → `ValueError`" test guards it | feat.mappings (section 3, section 6); tool:repo (`transform.py`) |
| R-T2 | Release of TSK-01 depends on feed v3 in production | Gates release, not coding; v2 and v3 both load | task.components (R-I1); feat.mappings (MQ-C) |
| R-T3 | Column spelling `paid_date` | The customer contract lists `paid_date`, so CQ-I1 (DE-04) is answered; close with MQ-A | feat.mappings (MQ-A); task.components (CQ-I1) |

## 6. Open questions (non-blocking)

| ID | Question | Interim answer | Owner (proposed) |
|---|---|---|---|
| TQ-1 | Carried MQ-B: the customer contract says `paid_date` is empty when unpaid, while the model keeps an unpaid record with a valid `paid_date`. Reject, clear, or keep? | Keep (TR-1 adds no such rule); `delays_by_vendor` ignores it because it filters on `paid` | Product / Architect; feed owner |
| TQ-2 | Carried MQ-C: must v2 and v3 be accepted side by side, and from what date is v3 effective? | Yes; TR-1 accepts both | Data engineering lead / feed owner |
| TQ-3 | The inputs do not give the target package version after `0.4.0` (patch or minor step) | Left to DE-08 under the TSK-01 budget | Data engineering lead |
| TQ-4 | Carried CQ-I2 / DQ-1: which production component fetches the feed and calls `pipeline.run` and `delays_by_vendor`? | Not needed for the transformation code; the end-to-end harness wires the stages | Architect |

## 7. Validation check

| Check | Result | Source |
|---|---|---|
| Existing sources and pipelines read | Pass: `ingest.py`, `transform.py`, `dq.py`, `pipeline.py`, `schedule.json`, `__init__.py`, `tests/test_pipeline.py` | tool:repo |
| Transformation sources/pipelines to create or update listed | Pass: update `transform.to_model` (TR-1), create `delays.py` (TR-2); nothing else | feat.mappings; task.components |
| Minimum lines of code | Pass: additive change in one existing file plus one new module; no change to ingestion, data quality, orchestration or schedule | sections 2 and 3 |
| Every mapping rule in feat.mappings has a component | Pass: section 2 rows → TR-1; section 4 → TR-2; section 5 → TR-6; section 6 → test budget | feat.mappings |
| Secrets in all environments | Pass: none required, none granted, no gap | task.components (section 4) |
| Conflict with an approved output | None | feat.mappings; task.components |
| Production or source-system data read | None; inputs, the tool view and synthetic rows only | record:brief |
| Blocking questions | None | — |